

SQL (Parte II)

Clase 04

IIC2413 - Sección 3

Prof. Cristian Riveros

Outline

Data Query Language (cont.)

Consultas anidadas

SQL / AR / Lógica

Outline

Data Query Language (cont.)

Consultas anidadas

SQL / AR / Lógica

DQL: El lenguaje de consultas de SQL

Q	:=	SELECT [DISTINCT] <lista-att>	}	Caso base
		FROM <lista-relaciones>		
		WHERE <condicion>		
		Q1 UNION Q2	}	Casos inductivos
		Q1 UNION ALL Q2		
		Q1 INTERSECT Q2		
		Q1 EXCEPT Q2		
		Q1 EXCEPT ALL Q2		

- SQL es **declarativo**.

*"Decimos lo que queremos,
pero sin dar detalles de cómo lo computamos"*

- El motor RDBMS transforma la consulta en un algoritmo eficiente.

SQL basado en **álgebra relacional** / **lógica de primer orden**

DQL: Selección + Proyección

SQL

```
SELECT [DISTINCT] <lista-att>  
FROM <relacion>  
WHERE <condicion>
```

Algebra relacional

$$\pi_{\langle \text{lista-att} \rangle} \left(\sigma_{\langle \text{condicion} \rangle} \left(\underbrace{R}_{\langle \text{relacion} \rangle} \right) \right)$$

Consultas de primer orden

$$Q(\underbrace{\bar{x}}_{\langle \text{lista-att} \rangle}) \leftarrow \exists \bar{y}. \underbrace{R(\bar{x}, \bar{y})}_{\langle \text{relacion} \rangle} \wedge \underbrace{\theta}_{\langle \text{condicion} \rangle}$$

Ejemplo en ejecución: sistema planetario

Planets

name	dist	radio	grav	years	temp	ring
Mercurio	0.39	0.38	2.8	0.241	440	FALSE
Venus	0.72	0.95	8.9	0.615	730	FALSE
Tierra	1.00	1.00	9.8	1.000	288	FALSE
Marte	1.52	0.53	3.7	1.880	186	FALSE
Júpiter	5.20	10.97	22.9	11.862	152	TRUE
Saturno	9.54	9.14	9.1	29.447	134	TRUE
Urano	19.19	3.98	7.8	84.017	76	TRUE
Neptuno	30.07	3.86	11.0	164.791	53	TRUE

Satellites

name	planet	discover	dyear
Luna	Tierra	--	--
Ganímedes	Júpiter	Galileo	1610
Calisto	Júpiter	Galileo	1610
Europa	Júpiter	Galileo	1610
Ío	Júpiter	Galileo	1610
Titán	Saturno	Huygens	1655
Triton	Neptuno	Lassell	1845

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

(ejemplo de Aidan Hogan – UChile)

DQL: Aprendiendo SQL con álgebra relacional

Álgebra relacional

1. R ✓
2. Proyección $\pi_{B_1, \dots, B_k}$ ✓
3. Selección $\sigma_{A \sim a}, \sigma_{A \sim B}$ ✓
4. Renombrar $\rho_{A \rightarrow B}$ ✓
5. Producto cruz $\times$
6. Join $\bowtie$
7. Operadores de conjuntos: $\cup$, $\cap$, $\setminus$

DQL: Producto cruz ×

Satellites

name	planet	discover	dyear
Luna	Tierra	--	--
Ganímedes	Júpiter	Galileo	1610
Calisto	Júpiter	Galileo	1610
Europa	Júpiter	Galileo	1610
Ío	Júpiter	Galileo	1610
Titán	Saturno	Huygens	1655
Tritón	Neptuno	Lassell	1846

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Ejemplo: $\pi_{\text{name, planet, spaceship}}(\text{Satellites} \times \text{Landings})$

SELECT name, planet, spaceship
FROM Satellites, Landings

name	planet	spaceship
Luna	?	Messenger
Luna	?	Venera3
...	...	...
Ganímedes	?	Messenger
Ganímedes	?	Venera3
...	...	...

¿cuál planet escoge SQL?

DQL: Producto cruz $\times$

Satellites

name	planet	discover	dyear
Luna	Tierra	--	--
Ganímedes	Júpiter	Galileo	1610
Calisto	Júpiter	Galileo	1610
Europa	Júpiter	Galileo	1610
Ío	Júpiter	Galileo	1610
Titán	Saturno	Huygens	1655
Tritón	Neptuno	Lassell	1846

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Definición (útil)

Si $\text{att}(R) = \{a_1, a_2, \dots, a_k\}$, entonces usaremos:

$$\rho_X(R) := \rho_{a_1 \rightarrow X.a_1}(\rho_{a_2 \rightarrow X.a_2}(\dots(\rho_{a_k \rightarrow X.a_k}(R))\dots))$$

Renombramos todos los atributos con el prefijo X .

DQL: Producto cruz ×

Satellites

name	planet	discover	dyear
Luna	Tierra	--	--
Ganímedes	Júpiter	Galileo	1610
Calisto	Júpiter	Galileo	1610
Europa	Júpiter	Galileo	1610
Ío	Júpiter	Galileo	1610
Titán	Saturno	Huygens	1655
Tritón	Neptuno	Lassell	1846

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Ejemplo: $\pi_{\text{name, S.planet, spaceship}}(\rho_S(\text{Satellites}) \times \text{Landings})$

SELECT name, S.planet, spaceship
FROM Satellites AS S, Landings

name	planet	spaceship
Luna	Tierra	Messenger
Luna	Tierra	Venera3
...	...	...
Ganímedes	Júpiter	Messenger
Ganímedes	Júpiter	Venera3
...	...	...

“AS S” en FROM renombra cada atributo A como S.A

DQL: Producto cruz $\times$

Satellites

name	planet	discover	dyear
Luna	Tierra	--	--
Ganímedes	Júpiter	Galileo	1610
Calisto	Júpiter	Galileo	1610
Europa	Júpiter	Galileo	1610
Ío	Júpiter	Galileo	1610
Titán	Saturno	Huygens	1655
Tritón	Neptuno	Lassell	1846

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Ejemplo: $\pi_{\text{name}, \text{S.planet}, \text{spaceship}}(\rho_{\text{S}}(\text{Satellites}) \times \text{Landings})$

```
SELECT name, S.planet, spaceship  $\equiv$   
FROM Satellites AS S, Landings
```

```
Q( $x_{\text{name}}, x_{\text{planet}}, x_{\text{spaceship}}$ )  $\leftarrow$   
 $\exists x \exists y \exists z \exists u \exists v. \text{Satellites}(x_{\text{name}}, x_{\text{planet}}, x, y) \wedge \text{Landings}(x_{\text{spaceship}}, z, u, v)$ 
```

¿cómo se escribe en lógica de primer orden?

DQL: Aprendiendo SQL con álgebra relacional

Álgebra relacional

1. R ✓
2. Proyección $\pi_{B_1, \dots, B_k}$ ✓
3. Selección $\sigma_{A \sim a}, \sigma_{A \sim B}$ ✓
4. Renombrar $\rho_{A \rightarrow B}$ ✓
5. Producto cruz $\times$ ✓
6. Join $\bowtie$
7. Operadores de conjuntos: $\cup$, $\cap$, $\setminus$

DQL: Join $\bowtie$ (recordatorio)

Si $\bar{r}_1$ y $\bar{r}_2$ son **compatibles** ($\bar{r}_1 \sim \bar{r}_2 : \forall A \in \text{att}(\bar{r}_1) \cap \text{att}(\bar{r}_2). \bar{r}_1(A) = \bar{r}_2(A)$):

$$[\bar{r}_1 \bowtie \bar{r}_2](A) := \begin{cases} \bar{r}_1(A) & \text{si } A \in \text{att}(\bar{r}_1) \\ \bar{r}_2(A) & \text{si } A \in \text{att}(\bar{r}_2) \end{cases}$$

Definición

Para **relaciones** $\mathcal{R}_1$ y $\mathcal{R}_2$ se define el **join**:

$$\mathcal{R}_1 \bowtie \mathcal{R}_2 := \{ \bar{r}_1 \bowtie \bar{r}_2 \mid \bar{r}_1 \in \mathcal{R}_1 \wedge \bar{r}_2 \in \mathcal{R}_2 \wedge \bar{r}_1 \sim \bar{r}_2 \}$$

Ejemplo de $\bowtie$

pid	name	cid		pid	job		pid	name	cid	job
1	Ana	101	$\bowtie$	1	Guía	=	1	Ana	101	Guía
2	Ben	102		2	Chef		2	Ben	102	Chef
3	Zoe	102		2	Guía		2	Ben	102	Guía
				2	Poeta		2	Ben	102	Poeta

DQL: Join ⋈

Satellites

name	planet	discover	dyear
Luna	Tierra	--	--
Ganímedes	Júpiter	Galileo	1610
Calisto	Júpiter	Galileo	1610
Europa	Júpiter	Galileo	1610
Ío	Júpiter	Galileo	1610
Titán	Saturno	Huygens	1655
Tritón	Neptuno	Lassell	1846

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Ejemplo: $\pi_{\text{name, planet}}(\text{Satellites} \bowtie \text{Landings})$

```
SELECT name, planet
FROM Satellites
      NATURAL JOIN Landings
```



name	planet
Ganímedes	Júpiter
Calisto	Júpiter
Europa	Júpiter
Ío	Júpiter

En general, este es el **operador de join** en SQL **menos usado** :(

DQL: Join ⋈

Satellites

name	planet	discover	dyear
Luna	Tierra	--	--
Ganímedes	Júpiter	Galileo	1610
Calisto	Júpiter	Galileo	1610
Europa	Júpiter	Galileo	1610
Ío	Júpiter	Galileo	1610
Titán	Saturno	Huygens	1655
Tritón	Neptuno	Lassell	1846

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Ejemplo: $\pi_{\text{name, planet}}(\text{Satellites} \bowtie \text{Landings})$

```
SELECT name, planet
FROM Satellites
      JOIN Landings USING (planet)
```



name	planet
Ganímedes	Júpiter
Calisto	Júpiter
Europa	Júpiter
Ío	Júpiter

Un poco más usado en SQL (pero no el más usado) :(

DQL: Join ⋈

Satellites

name	planet	discover	dyear
Luna	Tierra	--	--
Ganímedes	Júpiter	Galileo	1610
Calisto	Júpiter	Galileo	1610
Europa	Júpiter	Galileo	1610
Ío	Júpiter	Galileo	1610
Titán	Saturno	Huygens	1655
Tritón	Neptuno	Lassell	1846

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Teorema (recordatorio)

Para $\mathcal{R}_1$ y $\mathcal{R}_2$ con $\text{att}(\mathcal{R}_1) = \{A, B\}$ y $\text{att}(\mathcal{R}_2) = \{B, C\}$ se tiene que:

$$\mathcal{R}_1 \bowtie \mathcal{R}_2 = \pi_{A,B,C} \left(\sigma_{B=B'} \left(\mathcal{R}_1 \times \rho_{B \rightarrow B'}(\mathcal{R}_2) \right) \right)$$

Siempre podemos hacer un join con **producto cruz** y **selección**.

DQL: Join ⋈

Satellites

name	planet	discover	dyear
Luna	Tierra	--	--
Ganímedes	Júpiter	Galileo	1610
Calisto	Júpiter	Galileo	1610
Europa	Júpiter	Galileo	1610
Ío	Júpiter	Galileo	1610
Titán	Saturno	Huygens	1655
Tritón	Neptuno	Lassell	1846

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Ejemplo: $\pi_{\text{name, planet}}(\text{Satellites} \bowtie \text{Landings})$

```
SELECT name, S.planet
FROM Satellites AS S, Landings AS L
WHERE S.planet = L.planet
```



name	planet
Ganímedes	Júpiter
Calisto	Júpiter
Europa	Júpiter
Ío	Júpiter

Esta es la forma **más común** como los usuarios hacen un join.

DQL: Join ⋈

Satellites

name	planet	discover	dyear
Luna	Tierra	--	--
Ganímedes	Júpiter	Galileo	1610
Calisto	Júpiter	Galileo	1610
Europa	Júpiter	Galileo	1610
Ío	Júpiter	Galileo	1610
Titán	Saturno	Huygens	1655
Tritón	Neptuno	Lassell	1846

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Ejemplo: $\pi_{\text{name, planet}}(\text{Satellites} \bowtie \text{Landings})$

```
SELECT name, S.planet
FROM Satellites AS S, Landings AS L
WHERE S.planet = L.planet
```

$Q(x_{\text{name}}, x_{\text{planet}}) \leftarrow \exists x \exists y \exists z \exists u \exists v. \exists w. \underbrace{\text{Satellites}(x_{\text{name}}, x_{\text{planet}}, x, y) \wedge \text{Landings}(z, u, v, w)}_{\langle \text{condicion} \rangle} \wedge x_{\text{planet}} = u$

¿y en lógica de primer orden?

DQL: Join ⋈

Satellites

name	planet	discover	dyear
Luna	Tierra	--	--
Ganímedes	Júpiter	Galileo	1610
Calisto	Júpiter	Galileo	1610
Europa	Júpiter	Galileo	1610
Ío	Júpiter	Galileo	1610
Titán	Saturno	Huygens	1655
Tritón	Neptuno	Lassell	1846

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Ejemplo: $\pi_{\text{name, planet}}(\text{Satellites} \bowtie \text{Landings})$

```
SELECT name, S.planet
FROM Satellites AS S, Landings AS L
WHERE S.planet = L.planet
```

$$Q(x_{\text{name}}, x_{\text{planet}}) \leftarrow \exists x \exists y \exists z \exists v \exists w.$$
$$\text{Satellites}(x_{\text{name}}, x_{\text{planet}}, x, y) \wedge \text{Landings}(z, x_{\text{planet}}, v, w)$$

¿y en lógica de primer orden?

DQL: Join $\bowtie$ + Selecciones

Satellites

name	planet	discover	dyear
Luna	Tierra	--	--
Ganímedes	Júpiter	Galileo	1610
Calisto	Júpiter	Galileo	1610
Europa	Júpiter	Galileo	1610
Ío	Júpiter	Galileo	1610
Titán	Saturno	Huygens	1655
Tritón	Neptuno	Lassell	1846

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Ejemplo: $\pi_{\text{name, planet}}(\sigma_{\text{dyear} > 2000}(\text{Satellites} \bowtie \text{Landings}))$

```
SELECT name, S.planet
FROM Satellites AS S, Landings AS L
WHERE S.planet = L.planet
      AND dyear > 2000
```



name	planet
------	--------

DQL: Selección + Proyección + Joins

SQL

```
SELECT [DISTINCT] <lista-att>  
FROM <lista-relaciones>  
WHERE <condicion>
```

Algebra relacional

$$\pi_{\langle \text{lista-att} \rangle} \left(\sigma_{\langle \text{condicion} \rangle} \left(\underbrace{R_1 \times \dots \times R_k}_{\langle \text{lista-relaciones} \rangle} \right) \right)$$

Consultas de primer orden

$$\underbrace{Q(\bar{x})}_{\langle \text{lista-att} \rangle} \leftarrow \exists \bar{y}. \underbrace{R_1 \wedge \dots \wedge R_k}_{\langle \text{lista-relaciones} \rangle} \wedge \overbrace{\theta}^{\langle \text{condicion} \rangle}$$

DQL: Aprendiendo SQL con álgebra relacional

Álgebra relacional

1. R ✓
2. Proyección $\pi_{B_1, \dots, B_k}$ ✓
3. Selección $\sigma_{A \sim a}, \sigma_{A \sim B}$ ✓
4. Renombrar $\rho_{A \rightarrow B}$ ✓
5. Producto cruz $\times$ ✓
6. Join $\bowtie$ ✓
7. Operadores de conjuntos: $\cup$, $\cap$, $\setminus$

DQL: Unión $\cup$

Satellites

name	planet	discover	dyear
Luna	Tierra	--	--
Ganímedes	Júpiter	Galileo	1610
Calisto	Júpiter	Galileo	1610
Europa	Júpiter	Galileo	1610
Ío	Júpiter	Galileo	1610
Titán	Saturno	Huygens	1655
Tritón	Neptuno	Lassell	1846

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Ejemplo: $\pi_{\text{planet}}(\text{Satellites}) \cup \pi_{\text{planet}}(\text{Landings})$

```
SELECT planet
FROM Satellites
UNION
SELECT planet
FROM Landings
```

$\Rightarrow$

planet
Tierra
Júpiter
Saturno
Neptuno
Mercurio
Venus
Marte

¿y donde están los duplicados?

DQL: Unión U

Satellites

name	planet	discover	dyear
Luna	Tierra	--	--
Ganímedes	Júpiter	Galileo	1610
Calisto	Júpiter	Galileo	1610
Europa	Júpiter	Galileo	1610
Ío	Júpiter	Galileo	1610
Titán	Saturno	Huygens	1655
Tritón	Neptuno	Lassell	1846

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

SQL y duplicados

SELECT ⇒ Proyección con **duplicados**

UNION ⇒ Unión de conjuntos

UNION ALL ⇒ Unión con **duplicados**

DQL: Unión $\cup$

Satellites

name	planet	discover	dyear
Luna	Tierra	--	--
Ganímedes	Júpiter	Galileo	1610
Calisto	Júpiter	Galileo	1610
Europa	Júpiter	Galileo	1610
Ío	Júpiter	Galileo	1610
Titán	Saturno	Huygens	1655
Tritón	Neptuno	Lassell	1846

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Ejemplo: $\pi_{\text{planet}}(\text{Satellites}) \cup \pi_{\text{planet}}(\text{Landings})$

```
SELECT planet
FROM Satellites
UNION ALL
SELECT planet
FROM Landings
```

$\Rightarrow$

planet
Tierra
Júpiter
Júpiter
Júpiter
Júpiter
Saturno
...

DQL: Operadores de conjuntos: $\cup$, $\cap$, $\setminus$

Unión $\cup$

UNION $\Rightarrow$ Unión de conjuntos

UNION ALL $\Rightarrow$ Unión con **duplicados**

Intersección $\cap$

INTERSECT $\Rightarrow$ Intersección de conjuntos

INTERSECT ALL $\Rightarrow$ Intersección con **duplicados**

Diferencia $\setminus$

EXCEPT $\Rightarrow$ Diferencia de conjuntos

EXCEPT ALL $\Rightarrow$ Diferencia con **duplicados**

Outline

Data Query Language (cont.)

Consultas anidadas

SQL / AR / Lógica



Consultas anidadas en SQL

Tipos de consultas anidadas

1. Anidación en FROM
2. Anidación en WHERE
 - Operador IN
 - Operador EXISTS
 - Por valor

Anidación en FROM

Planets			Landings			
name	grav	...	spaceship	planet	ctry	year
Mercurio	2.8	...	Messenger	Mercurio	EEUU	2015
Venus	8.9	...	Venera 3	Venus	URSS	1966
Tierra	9.8	...	Pioneer	Venus	EEUU	1978
Marte	3.7	...	Mars 2	Marte	URSS	1971
Júpiter	22.9	...	Viking 1	Marte	EEUU	1976
Saturno	9.1	...	Beagle 2	Marte	ESA	2003
Urano	7.8	...	Galileo	Júpiter	EEUU	2003
Neptuno	11.0	...				

Ejemplo con anidación en FROM

```
SELECT spaceship, planet
FROM Landings,
  ( SELECT name
    FROM Planets
    WHERE grav >= 9.8 ) AS P
WHERE planet = P.name
```



spaceship	planet
Galileo	Júpiter

Anidación en FROM

Ejemplo con anidación en FROM

```
SELECT spaceship, planet
FROM Landings,
    ( SELECT name
      FROM Planets
      WHERE grav >= 9.8 ) AS P
WHERE planet = P.name
```



spaceship	planet
Galileo	Júpiter

¿cómo hacemos esta consulta en álgebra relacional?

$$P \leftarrow \pi_{\text{name}}(\sigma_{\text{grav} \geq 9.8}(\text{Planets}))$$
$$\text{out} \leftarrow \pi_{\text{spaceship, planet}}(\sigma_{\text{planet} = P.\text{name}}(\text{Planets} \times \rho_P(P)))$$

Anidación en FROM

Ejemplo con anidación en FROM

```
SELECT spaceship, planet
FROM Landings,
    ( SELECT name
      FROM Planets
      WHERE grav >= 9.8 ) AS P
WHERE planet = P.name
```



spaceship	planet
Galileo	Júpiter

¿cómo hacemos esta consulta en lógica de primer orden?

(omitimos atributos por simplificación)

$$P(n) \leftarrow \exists g. \overbrace{\text{Planets}(n, g)} \wedge g \geq 9.8$$

$$Q(s, p) \leftarrow \exists c \exists y \exists n. \text{Landings}(s, p, c, y) \wedge P(n) \wedge p = n$$

Anidación en FROM

Ejemplo con anidación en FROM

```
SELECT spaceship, planet
FROM Landings,
    ( SELECT name
      FROM Planets
      WHERE grav >= 9.8 ) AS P
WHERE planet = P.name
```



spaceship	planet
Galileo	Júpiter

¿cómo hacemos esta consulta en lógica de primer orden?

$$P(n) \leftarrow \exists g. \text{Planets}(n, g) \wedge g \geq 9.8$$

$$Q(s, p) \leftarrow \exists c \exists y \exists n. \text{Landings}(s, p, c, y) \wedge P(n) \wedge p = n$$

Equivalente a una **subexpresión** / **subformula** en AR / lógica.

Anidación en WHERE / IN

Planets			Landings			
name	grav	...	spaceship	planet	ctry	year
Mercurio	2.8	...	Messenger	Mercurio	EEUU	2015
Venus	8.9	...	Venera 3	Venus	URSS	1966
Tierra	9.8	...	Pioneer	Venus	EEUU	1978
Marte	3.7	...	Mars 2	Marte	URSS	1971
Júpiter	22.9	...	Viking 1	Marte	EEUU	1976
Saturno	9.1	...	Beagle 2	Marte	ESA	2003
Urano	7.8	...	Galileo	Júpiter	EEUU	2003
Neptuno	11.0	...				

Ejemplo con anidación en IN

```
SELECT spaceship, planet
FROM Landings
WHERE planet IN
    ( SELECT name
      FROM Planets
      WHERE grav >= 9.8 )
```



spaceship	planet
Galileo	Júpiter

Anidación en WHERE / IN

Ejemplo con anidación en IN

```
SELECT spaceship, planet
FROM Landings
WHERE planet IN
    ( SELECT name
      FROM Planets
      WHERE grav >= 9.8 )
```



spaceship	planet
Galileo	Júpiter

¿cómo hacemos esta consulta en lógica de primer orden?

$$N(n) \leftarrow \exists g. \text{Planets}(n, g) \wedge g \geq 9.8$$

$$Q(s, p) \leftarrow \exists c \exists y. \text{Landings}(s, p, c, y) \wedge N(p)$$

¿cómo hacemos esta consulta en **álgebra relacional**?

Anidación en WHERE / NOT IN

Ejemplo con anidación en NOT IN

```
SELECT spaceship, planet
FROM Landings
WHERE planet NOT IN
    ( SELECT name
      FROM Planets
      WHERE grav >= 9.8 )
```



spaceship	planet
Messenger	Mercurio
Venera 3	Venus
Pioneer	Venus
Mars 2	Marte
Viking 1	Marte
Beagle 2	Marte

¿cómo hacemos esta consulta en lógica de primer orden?

$$N(n) \leftarrow \exists g. \text{Planets}(n, g) \wedge g \geq 9.8$$

$$Q(s, p) \leftarrow \exists c \exists y. \text{Landings}(s, p, c, y) \wedge \neg N(p)$$

¿cómo hacemos esta consulta en **álgebra relacional**?

Anidación en WHERE / NOT IN

Ejemplo con anidación en NOT IN

```
SELECT spaceship, planet
FROM Landings
WHERE planet NOT IN
    ( SELECT name
      FROM Planets
      WHERE grav >= 9.8 )
```



spaceship	planet
Messenger	Mercurio
Venera 3	Venus
Pioneer	Venus
Mars 2	Marte
Viking 1	Marte
Beagle 2	Marte

¿cómo hacemos esta consulta en álgebra relacional?

$$\bar{P} \leftarrow \pi_{\text{name}}(\sigma_{\text{grav} < 9.8}(\text{Planets}))$$
$$\text{out} \leftarrow \pi_{\text{spaceship, planet}}(\sigma_{\text{planet} = \text{name}}(\text{Planets} \times \bar{P}))$$

OJO: **no siempre** es tan simple simular NOT IN en AR.

Anidación en WHERE / EXISTS

Planets		
name	dist	...
Mercurio	0.39	...
Venus	0.72	...
Tierra	1.00	...
Marte	1.52	...
Júpiter	5.20	...
Saturno	9.54	...
Urano	19.19	...
Neptuno	30.07	...

Landings			
spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Ejemplo con anidación en EXISTS

```
SELECT name, dist
FROM Planets
WHERE EXISTS
    ( SELECT *
      FROM Landings
      WHERE year >= 1976 )
```



name	dist
Mercurio	0.39
Venus	0.72
Tierra	1.00
Marte	1.52
Júpiter	5.20
Saturno	9.54
Urano	19.19
Neptuno	30.07

No es muy interesante :(

Anidación en WHERE / EXISTS + correlación

Planets		
name	dist	...
Mercurio	0.39	...
Venus	0.72	...
Tierra	1.00	...
Marte	1.52	...
Júpiter	5.20	...
Saturno	9.54	...
Urano	19.19	...
Neptuno	30.07	...

Landings			
spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Ejemplo con anidación en EXISTS

```
SELECT name, dist
FROM Planets
WHERE EXISTS
  ( SELECT *
    FROM Landings
    WHERE year >= 1976 AND
          name = planet )
      correlación
```



name	dist
Mercurio	0.39
Venus	0.72
Marte	1.52
Júpiter	5.20

Anidación en WHERE / EXISTS + correlación

Ejemplo con anidación en EXISTS

```
SELECT name, dist
FROM Planets
WHERE EXISTS
  ( SELECT *
    FROM Landings
    WHERE year >= 1976 AND
          name = planet )
```



name	dist
Mercurio	0.39
Venus	0.72
Marte	1.52
Júpiter	5.20

¿cómo hacemos esta consulta en lógica de primer orden?

$$E(x) \leftarrow \exists s \exists p \exists c \exists y. \text{Landings}(s, p, c, y) \wedge y \geq 1976 \wedge x = p$$
$$Q(n, d) \leftarrow \text{Planets}(n, d) \wedge E(n)$$

La variable correlacionada es **expuesta** en la subconsulta

Anidación en WHERE / EXISTS + correlación

Ejemplo con anidación en EXISTS

```
SELECT name, dist
FROM Planets
WHERE EXISTS
  ( SELECT *
    FROM Landings
    WHERE year >= 1976 AND
          name = planet )
```



name	dist
Mercurio	0.39
Venus	0.72
Marte	1.52
Júpiter	5.20

¿cómo hacemos esta consulta en álgebra relacional?

Ejercicio: encuentre la consulta en AR.

Anidación en WHERE / EXISTS + correlación

Ejemplo con anidación en EXISTS

```
SELECT name, dist
FROM Planets
WHERE EXISTS
  ( SELECT *
    FROM Landings
    WHERE year >= 1976 AND
          name = planet )
```



name	dist
Mercurio	0.39
Venus	0.72
Marte	1.52
Júpiter	5.20

¿son equivalentes?

```
SELECT name, dist
FROM Planets, Landings
WHERE year >= 2000 AND name = planet
```



name	dist
Mercurio	0.39
Venus	0.72
Marte	1.52
Marte	1.52
Júpiter	5.20

Al desanidar una consulta debemos tener cuidado con los **duplicados**

Anidación en WHERE / NOT EXISTS + correlación

Planets		
name	dist	...
Mercurio	0.39	...
Venus	0.72	...
Tierra	1.00	...
Marte	1.52	...
Júpiter	5.20	...
Saturno	9.54	...
Urano	19.19	...
Neptuno	30.07	...

Landings			
spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Ejemplo con anidación en NOT EXISTS

```
SELECT name, dist
FROM Planets
WHERE NOT EXISTS
  ( SELECT *
    FROM Landings
    WHERE year >= 1976 AND
          name = planet )
correlación
```

⇒

name	dist
Tierra	1.0
Saturno	9.54
Urano	19.19
Neptuno	30.07

Anidación en WHERE / NOT EXISTS + correlación

Ejemplo con anidación en NOT EXISTS

```
SELECT name, dist
FROM Planets
WHERE NOT EXISTS
  ( SELECT *
    FROM Landings
    WHERE year >= 1976 AND
          name = planet )
      correlación
```

⇒

name	dist
Tierra	1.0
Saturno	9.54
Urano	19.19
Neptuno	30.07

¿cómo hacemos esta consulta en lógica de primer orden?

$$E(x) \leftarrow \exists s \exists p \exists c \exists y. \text{Landings}(s, p, c, y) \wedge y \geq 1976 \wedge x = p$$

$$Q(n, d) \leftarrow \text{Planets}(n, d) \wedge \neg E(n)$$

¿cómo lo hacemos en **álgebra relacional**? (no es tan directo siempre)

Anidación en WHERE / por valor

Planets

name	dist	radio	grav	years	temp	ring
Mercurio	0.39	0.38	2.8	0.241	440	FALSE
Venus	0.72	0.95	8.9	0.615	730	FALSE
Tierra	1.00	1.00	9.8	1.000	288	FALSE
Marte	1.52	0.53	3.7	1.880	186	FALSE
Júpiter	5.20	10.97	22.9	11.862	152	TRUE
Saturno	9.54	9.14	9.1	29.447	134	TRUE
Urano	19.19	3.98	7.8	84.017	76	TRUE
Neptuno	30.07	3.86	11.0	164.791	53	TRUE

Ejemplo con anidación por valor

```
SELECT name
FROM Planets AS P1
WHERE P1.grav >
  ( SELECT P2.grav
    FROM Planets AS P2
    WHERE P2.name = 'Tierra' )
```



name
Júpiter
Neptuno

Si la subconsulta no entrega un **único resultado**, puede dar **error**

Anidación en WHERE / por valor + ANY/ALL

Planets						
name	dist	radio	grav	years	temp	ring
Mercurio	0.39	0.38	2.8	0.241	440	FALSE
Venus	0.72	0.95	8.9	0.615	730	FALSE
Tierra	1.00	1.00	9.8	1.000	288	FALSE
Marte	1.52	0.53	3.7	1.880	186	FALSE
Júpiter	5.20	10.97	22.9	11.862	152	TRUE
Saturno	9.54	9.14	9.1	29.447	134	TRUE
Urano	19.19	3.98	7.8	84.017	76	TRUE
Neptuno	30.07	3.86	11.0	164.791	53	TRUE

Ejemplo con anidación por valor + ANY / ALL

```
SELECT name
FROM Planets AS P1
WHERE P1.grav > ANY
  ( SELECT P2.grav
    FROM Planets AS P2
    WHERE P2.dist > 1.0 )
```

```
SELECT name
FROM Planets AS P1
WHERE P1.grav > ALL
  ( SELECT P2.grav
    FROM Planets AS P2
    WHERE P2.dist > 1.0 )
```

Importante: ANY/ALL no está soportado en sqlite.

Outline

Data Query Language (cont.)

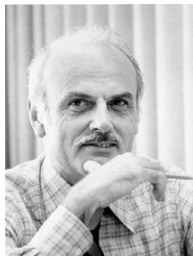
Consultas anidadas

SQL / AR / Lógica

¿cuál es la relación de SQL con AR / FO?

Teorema de Codd

Álgebra relacional y lógica de primer orden son **igualmente expresivos**.
En otras palabras, para toda consulta de primer orden,
existe una expresión **equivalente** en álgebra relacional y viceversa.



Edgard Codd

Relational Completeness of
Data Base Sublanguages

Data Base Systems. 1972.

Demostración: ver libro *Database Theory*.

¿cuál es la relación de SQL con AR / FO?

Teorema de Codd

Álgebra relacional y lógica de primer orden son **igualmente expresivos**.

En otras palabras, para toda consulta de primer orden, existe una expresión **equivalente** en álgebra relacional y viceversa.

$$\text{SQL}^* = \text{Álgebra Relacional} = \text{Lógica de primer orden}^\dagger$$

* Al fragmento de SQL que vimos hasta ahora.

† *domain-independent*.
(como fue definida en este curso)

¿cuál es la relación de SQL con AR / FO?

Teorema de Codd

Álgebra relacional y lógica de primer orden son **igualmente expresivos**.
En otras palabras, para toda consulta de primer orden,
existe una expresión **equivalente** en álgebra relacional y viceversa.

$$\text{SQL}^* = \text{Álgebra Relacional} = \text{Lógica de primer orden}^\dagger$$

Consecuencias

- El diseño de SQL esta basado en **lógica de primer orden**.
- Un motor convierte las consultas en SQL al **álgebra relacional** para hacer algoritmos eficientes de evaluación.
- Podemos estudiar **lógica de primer orden** para entender SQL.

¿cuál es la relación de SQL con AR / FO?

SELECT-FROM-WHERE

```
SELECT [DISTINCT] <lista-att>  
FROM <lista-relaciones>  
WHERE <condicion>
```

Proyección-selección-join

$$\pi_{\langle \text{lista-att} \rangle} \left(\sigma_{\langle \text{condicion} \rangle} \left(\underbrace{R_1 \times \dots \times R_k}_{\langle \text{lista-relaciones} \rangle} \right) \right)$$

Consultas conjuntivas (CQ)

$$\underbrace{Q(\bar{x})}_{\langle \text{lista-att} \rangle} \leftarrow \exists \bar{y}. \underbrace{R_1 \wedge \dots \wedge R_k}_{\langle \text{lista-relaciones} \rangle} \wedge \overbrace{\theta}^{\langle \text{condicion} \rangle}$$

CQ son las consultas **más estudiadas** en el área